Project Scenario: Online Retail Store

Objective

The goal of this project is to create a database for an online retail store that manages information about customers, orders, products, and order details. This project will demonstrate how to use different types of JOINs to retrieve related data from multiple tables.

Step 1: Database Design

We will create the following tables:

1. **Customers**: To store customer information.
2. **Products**: To store product details.
3. **Orders**: To store order information.
4. **OrderDetails**: To store information about products in each order.

Table Structure

1. **Customers Table**
    - `CustomerID` (INT, Primary Key, Auto-increment)
    - `FullName` (NVARCHAR(100), NOT NULL)
    - `Email` (NVARCHAR(100), NOT NULL)
2. **Products Table**
    - `ProductID` (INT, Primary Key, Auto-increment)
    - `ProductName` (NVARCHAR(100), NOT NULL)
    - `Price` (DECIMAL(10, 2), NOT NULL)
3. **Orders Table**
    - `OrderID` (INT, Primary Key, Auto-increment)
    - `CustomerID` (INT, Foreign Key referencing Customers)
    - `OrderDate` (DATE NOT NULL)
4. **OrderDetails Table**
    - `OrderDetailID` (INT, Primary Key, Auto-increment)
    - `OrderID` (INT, Foreign Key referencing Orders)
    - `ProductID` (INT, Foreign Key referencing Products)
    - `Quantity` (INT, NOT NULL)

Step 2: SQL Commands

Create the Database

```
CREATE DATABASE RetailStoreDB;
GO

USE RetailStoreDB;
GO
```

Create the Tables

Create Customers Table

```
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY IDENTITY(1,1),
    FullName NVARCHAR(100) NOT NULL,
    Email NVARCHAR(100) NOT NULL UNIQUE
);
GO
```

Create Products Table

```
CREATE TABLE Products (
    ProductID INT PRIMARY KEY IDENTITY(1,1),
    ProductName NVARCHAR(100) NOT NULL,
    Price DECIMAL(10, 2) NOT NULL
);
GO
```

Create Orders Table

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY IDENTITY(1,1),
    CustomerID INT,
    OrderDate DATE NOT NULL,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
GO
```

Create OrderDetails Table

```
CREATE TABLE OrderDetails (
    OrderDetailID INT PRIMARY KEY IDENTITY(1,1),
    OrderID INT,
    ProductID INT,
    Quantity INT NOT NULL,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
GO
```

Step 3: Insert Sample Data

Insert Customers

```
INSERT INTO Customers (FullName, Email)
VALUES
('Alice Johnson', 'alice@example.com'),
('Bob Smith', 'bob@example.com'),
('Charlie Brown', 'charlie@example.com');
GO
```

Insert Products

```
INSERT INTO Products (ProductName, Price)
VALUES
('Laptop', 999.99),
('Smartphone', 499.99),
('Tablet', 299.99);
GO
```

Insert Orders

```
INSERT INTO Orders (CustomerID, OrderDate)
VALUES
(1, '2024-09-01'),  -- Alice's order
(2, '2024-09-02');  -- Bob's order
GO
```

Insert OrderDetails

```
INSERT INTO OrderDetails (OrderID, ProductID, Quantity)
VALUES
(1, 1, 1),  -- Alice ordered 1 Laptop
(1, 2, 2),  -- Alice ordered 2 Smartphones
(2, 3, 1);  -- Bob ordered 1 Tablet
GO
```

Step 4: Using JOINs to Retrieve Data

1. INNER JOIN

Retrieve all orders along with customer names.

```
SELECT O.OrderID, C.FullName, O.OrderDate
FROM Orders O
INNER JOIN Customers C ON O.CustomerID = C.CustomerID;
GO
```

2. LEFT JOIN

Retrieve all customers and their orders, including those who haven't placed any orders.

```
SELECT C.FullName, O.OrderID, O.OrderDate
FROM Customers C
LEFT JOIN Orders O ON C.CustomerID = O.CustomerID;
GO
```

3. RIGHT JOIN

Retrieve all products and the orders they are associated with, including products that have not been ordered.

```
SELECT P.ProductName, O.OrderID, O.OrderDate
```

```
FROM Products P
RIGHT JOIN OrderDetails OD ON P.ProductID = OD.ProductID
RIGHT JOIN Orders O ON OD.OrderID = O.OrderID;
GO
```

4. FULL OUTER JOIN

Retrieve all customers, orders, and products, including those without a match in any table.

```
SELECT C.FullName, O.OrderID, P.ProductName
FROM Customers C
FULL OUTER JOIN Orders O ON C.CustomerID = O.CustomerID
FULL OUTER JOIN OrderDetails OD ON O.OrderID = OD.OrderID
FULL OUTER JOIN Products P ON OD.ProductID = P.ProductID;
GO
```

Conclusion

In this project, we successfully created a `RetailStoreDB` database, defined tables for customers, products, orders, and order details, and inserted sample data.